

GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

LABORATORY REPORT



AGILE SOFTWARE DEVELOPMENT LAB

Lab No. 1

BESE III YEAR, VI-SEM

Submitted By:

Prabin Thapa

Submitted To:

Er. Rajendra Bahadur Thapa

Date: July 4, 2026

NAME OF EXPERIMENT: Implementation of Source code management using git commands thorough github

AIM: To understand and implement Source Code Management (SCM) using Git and GitHub by performing essential version control operations such as repository creation, cloning, staging, committing, pushing, pulling, branching, merging, and collaboration.

Objectives:

1. To understand the concept of Source Code Management (SCM).
2. To learn the basic architecture and workflow of Git.
3. To create and manage a local Git repository.
4. To connect a local repository with a GitHub remote repository.
5. To perform common Git operations such as init, clone, add, commit, push, pull, status, branch, and merge.
6. To understand collaborative software development using GitHub.
7. To maintain different versions of source code efficiently.

THEORY:

Introduction to Source Code Management (SCM)

Source Code Management (SCM), also known as Version Control System (VCS), is a software tool that helps developers manage changes made to source code over time. It records every modification so that developers can compare versions, restore previous versions, and collaborate effectively with team members.

SCM eliminates problems such as accidental code deletion, conflicting modifications, and difficulty in tracking project history.

What is Git?

Git is a distributed version control system developed by Linus Torvalds in 2005 for Linux kernel development. Unlike centralized systems, every developer possesses a complete copy of the repository, including the entire project history.

Git offers:

- Fast performance
- Distributed development
- Data integrity
- Easy branching and merging
- Offline work capability
- Open-source availability

What is GitHub?

GitHub is a cloud-based hosting platform for Git repositories. It enables developers to:

- Store repositories online
- Collaborate with team members
- Track issues
- Review code through Pull Requests
- Manage software projects
- Maintain project documentation

GitHub acts as a remote repository where multiple developers can contribute simultaneously.

Git Architecture

The Git workflow mainly consists of three areas:

1. Working Directory

The directory where developers create or modify project files.

2. Staging Area (Index)

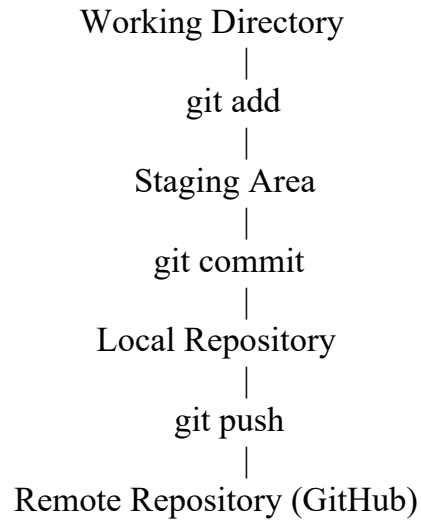
Temporary storage where selected changes are prepared before committing.

3. Local Repository

Stores committed project history locally.

4. Remote Repository (GitHub)

Stores project history online for collaboration.



Algorithm

Algorithm: Source Code Management Using Git Through GitHub

1. Install Git on the system.
2. Create a GitHub account.
3. Create a new repository on GitHub.
4. Open the terminal or command prompt.
5. Navigate to the project directory.
6. Initialize the local repository using `git init`.
7. Create project files.
8. Check repository status using `git status`.
9. Stage the files using `git add ..`
10. Commit the changes using `git commit -m "Initial Commit"`.
11. Connect the local repository with GitHub using `git remote add origin`.
12. Push the project using `git push -u origin main`.
13. Verify that files appear in the GitHub repository.
14. Modify project files when required.
15. Repeat the add, commit, and push process for future updates.
16. Pull changes from GitHub using `git pull` whenever collaborating with others.

17. Create branches for new features and merge them after testing.

18. End.

CODE:

➤ Initialize Repository

```
git init
```

Creates a new Git repository.

➤ Clone Repository

```
git clone <repository_url>
```

Downloads an existing GitHub repository.

➤ Check Repository Status

```
git status
```

Displays modified, staged, and untracked files.

➤ Stage Files

```
git add filename
```

or

```
git add .
```

Moves changes to the staging area.

➤ Commit Changes

```
git commit -m "Initial Commit"
```

Stores staged changes permanently in the repository.

➤ Connect Remote Repository

```
git remote add origin <repository_url>
```

Links the local repository with GitHub.

➤ Push Changes

```
git push origin main
```

Uploads commits to GitHub.

➤ Pull Changes

```
git pull origin main
```

Downloads the latest changes from GitHub.

➤ Create Branch

```
git branch feature
```

Creates a new development branch.

➤ Switch Branch

```
git checkout feature
```

or

```
git switch feature
```

Moves to another branch.

➤ Merge Branch

```
git merge feature
```

Combines changes into the current branch.

➤ View Commit History

```
git log
```

Displays the complete commit history.

RESULT:

The implementation of Source Code Management using Git and GitHub was successfully performed. The repository was initialized, files were committed, connected to GitHub, and synchronized using Git commands. Basic version control operations such as staging, committing, pushing, pulling, branching, and merging were successfully executed.

CONCLUSION:

This experiment demonstrated the practical implementation of Source Code Management using Git integrated with GitHub. Git enables efficient version tracking, rollback, branching, and collaboration, while GitHub provides a centralized platform for remote repository hosting and teamwork. By performing essential Git operations such as initialization, staging, committing, pushing, pulling, branching, and merging, the experiment highlighted how version control improves software development workflows. Overall, Git and GitHub provide a reliable, secure, and scalable solution for managing source code in both individual and collaborative software development projects.